

Candidate Assignment

Real-Time Financial Dashboard with Dynamic RBAC

This assignment is designed to evaluate prompt engineering, API integration, security implementation (RBAC), and code verification under realistic time constraints.

Objective

Build a full-stack, real-time Financial Market Dashboard that consumes public financial data APIs, implements role-based access control (RBAC), and dynamically gates UI elements and API endpoints based on user privileges.

- Target Time to Complete: 1–2 Hours
- AI Usage: Strongly encouraged. Candidates are expected to leverage AI to scaffold, code, refactor, and generate tests quickly.

Technical Requirements

1. Data Ingestion & Integration (Public API)

- Integrate the free CoinGecko API to fetch cryptocurrency data.
- Display live metrics (e.g., Asset Prices, 24h Volatility, FX Conversion Rates, Market Cap), with UI controls, dropdowns, or selectors allowing users to configure which specific live metrics to display.
- Handle API rate-limiting gracefully via a caching layer, mock fallback, or exponential retry handling.

2. Role-Based Access Control (RBAC) Architecture

Implement an authentication engine (JWT or mock session token) with three distinct roles:

Role	Access Level & Privileges
Viewer	Read-only access to standard metrics and market summaries.
Trader	Access to standard metrics + ability to create, edit, or remove assets in a custom watchlist.
Admin	Full access: View, edit watchlists, view system logs, and adjust global dashboard thresholds (e.g., market volatility alert triggers).

Enforcement Requirements

- **Backend Enforcer:** Middleware must enforce these permissions across all server routes/API endpoints (returning HTTP **401 Unauthorized** or **403 Forbidden** status codes on unauthorized requests).
- **Frontend Enforcer:** The UI must visually disable or hide components, buttons, and views depending on the authenticated role.

3. Core Features & UI

- **Role Switcher Widget:** A UI toggle/select input to rapidly swap authenticated users (e.g., "Log in as Viewer", "Log in as Trader", "Log in as Admin") for testing purposes.
- **Interactive Financial Charts:** At least one dynamic visual chart (Chart.js, Recharts, or D3) displaying historic or live trends.
- **Audit & Activity Log (Admin Only):** A viewable audit stream tracking user actions (e.g., Trader X added BTC to watchlist, Admin Y updated threshold).

Deliverables & Submission Guidelines

1. **Source Code Repository** (GitHub link) containing:
 - Frontend and Backend implementation.

- A `docker-compose.yml` file OR single command startup script (`npm run start:all`, `make dev`, etc.).
- 2. **AI_PROMPTS.md Log:** A brief document detailing:
 - Key prompts used to scaffold or solve complex parts of the project.
 - At least **one example** of where the AI generated incorrect/insecure code (a bug, hallucination, or RBAC flaw) and how you caught and fixed it.
- 3. **Automated Tests:**
 - An automated unit/integration test suite verifying the implementation.